SecureLAN: Zero Trust Communication Suite

Inline TLS/mTLS Inspection Proxy, Real-Time DLP & Zero Trust Access Control

Zscaler Skybound Engineering Project Report & Architecture Guide

1. Executive Summary & Architecture Overview

SecureLAN is a high-performance, real-time collaboration suite featuring **video conferencing, N-1 mixed audio streaming, screen sharing, group chat, and chunked file transfer**. The system has been architecturally hardened with an enterprise-grade **Zero Trust Architecture (ZTNA)**, **Inline TLS/mTLS Inspection Proxy**, and **Real-Time Data Loss Prevention (DLP)** engine directly inspired by **Zscaler's Zero Trust Exchange**.

Instead of unencrypted direct connections, client traffic routes through a single-pass inspection proxy that terminates TLS, evaluates device posture and RBAC, executes Deep Packet Inspection (DPI) on raw protocol frames to sanitize data in-flight, and re-originates an encrypted session to the origin server.

2. Original Baseline Capabilities (What Already Existed)

The foundation of the project comprised 5 core real-time communication modules operating across dedicated network ports:

Module	Port & Protocol	Baseline Implementation Details
Video Conferencing	UDP Port 5003	Selective Forwarding Unit (SFU) broadcasting OpenCV webcam captures compressed into JPEG payloads with struct-packed 8-byte session headers.
Audio Conferencing	UDP Port 5002	Full-duplex microphone capture via PyAudio with NumPy N-1 matrix mixing (each user hears all participants except their own echo).
Screen Sharing	TCP Port 5000	Ultra-fast desktop screen grabbing via `mss`, compressed to JPEG frames and streamed over TCP.
Group Text Chat	TCP Port 5000	Custom framed binary protocol with 4-byte magic number (`0xDEADBEEF`), 16-byte struct header, and JSON text payloads.

File Transfer	TCP Port 5001	Dedicated file transfer service supporting chunked streaming uploads and downloads with progress tracking and disk storage in `uploads/`.

Original Security Gaps: Prior to enhancement, traffic was unencrypted plaintext (exposed to Wireshark sniffing), authentication accepted any dummy username, access was unrestricted without RBAC, and no content/DLP inspection or compliance audit logging was present.

3. What Was Added (The Enterprise Security Package)

To eliminate security gaps and align directly with Zscaler's cloud security model, the `security/` package and `certs/` PKI was engineered:

Security Module	File Location	Core Enterprise Functionality
Automated PKI Generator	`security/cert_generator.py`	Dynamically generates an X.509 Root CA (RSA-4096) and signs leaf certificates (RSA-2048) for Server, Client, and Proxy with SANs.
TLS/mTLS Configuration	`security/tls_config.py`	Factory enforcing TLS 1.2+ minimum, modern cipher suites, and mutual TLS (mTLS) context generation for both client and proxy.
Zero Trust IAM & RBAC	`security/auth.py`	Stateless JWT tokens (HMAC-SHA256, 1-hr TTL), 4-tier RBAC (`ADMIN`, `PRESENTER`, `MEMBER`, `GUEST`), device trust scoring (0.0 - 1.0), and token revocation lists.
Inline DLP Engine	`security/dlp_engine.py`	9 regex patterns with Luhn validation (Credit Cards, SSNs, Aadhaar, AWS Keys, Passwords, API Keys), in-flight text masking, and magic-byte file validation to

		prevent extension spoofing.
Security Audit Logger	`security/audit_logger.py`	SOC 2 / ISO 27001 structured JSON-Lines logger (`security_audit.jsonl`), thread-safe in-memory circular buffer (deque), and real-time threat metrics.
Inline TLS Proxy (Flagship)	`security/tls_proxy.py`	Zscaler Zero Trust Exchange mirror: Dual-handshake proxy terminating client TLS, performing DPI on binary frames, redacting data in-memory, and re-encrypting upstream.

4. Step-by-Step Execution Guide

Follow these steps to run the complete architecture:

Step 1: Install Dependencies

```
pip install -r requirements.txt
```

Step 2: Start Origin Server (Terminal 1)

```
python main_server.py
```

Step 3: Start Inline TLS Inspection Proxy (Terminal 2)

```
python -m security.tls_proxy 5443 5000
```

Step 4: Start Client Interface (Terminal 3)

```
python client.py
```

5. Instant Interview CLI Demos

Execute these one-liners in a terminal during technical interviews to showcase live capability:

Demo 1: Real-Time DLP Engine with Luhn Algorithm Redaction

```
python -c "from security.dlp_engine import DLPEngine; e = DLPEngine(); res =
e.scan_text('Transfer to card 4111-1111-1111-1111 with key AKIAIOSFODNN7EXAMPLE');
print('Action:', res.action); print('Sanitized:', res.redacted_text)"
```

Demo 2: Zero Trust Authentication & Device Posture Scoring

```
python -c "from security.auth import AuthEngine, Permission; a = AuthEngine(); t =
a.authenticate('alice', 'pass123',
device_info={'os_name':'Windows','os_version':'11','hostname':'PC','ip_address':'127.0.0.1'}
); u = a.validate_token(t); print('User:', u.username, '| Role:', u.role.value, '| Trust
Score:', u.device_context.trust_score); print('Can Kick Users:', a.check_permission(u,
Permission.KICK_USER))"
```

Demo 3: Live Security Audit Telemetry

```
python -c "from security.audit_logger import AuditLogger;
print(AuditLogger().get_statistics())"
```

6. Resume Bullet Points & Interview Talking Points

Resume Project Title:

Zero Trust Secure LAN Collaboration Platform with Inline TLS Inspection & DLP

Technologies: Python, asyncio, TLS/mTLS, PyJWT, Cryptography (X.509), Deep Packet Inspection (DPI), PySide6, UDP SFU

- **Zscaler-Inspired Inline TLS Proxy:** Engineered an inline TLS inspection proxy mirroring Zscaler's Zero Trust Exchange — terminating client TLS, executing Deep Packet Inspection (DPI) on binary protocol frames, and re-encrypting upstream sessions.
- **Zero Trust & RBAC Engine:** Implemented stateless JWT authentication with fine-grained RBAC across 4 user tiers, calculating device posture trust scores and enforcing token revocation blacklists.
- **Real-Time Data Loss Prevention (DLP):** Developed an inline DLP scanning engine with 9 pattern recognizers and Luhn algorithm validation (credit cards, SSNs, AWS credentials) with in-flight payload masking and binary magic-byte inspection preventing extension-spoofing malware.
- **SOC 2 Compliant Audit Trail:** Designed an asynchronous structured audit logger producing immutable JSONL trails and an in-memory ring buffer for low-latency operational threat metrics.

- **High-Concurrency Multi-Protocol Media Core:** Designed a custom binary protocol handling 5 concurrent real-time services across 4 dedicated TCP/UDP ports, including an SFU-based UDP video forwarder, N-1 audio mixing with NumPy, and chunked TCP file transfer.

Interview Elevator Pitch:

"My project is a secure LAN collaboration suite designed specifically around Zscaler's architectural philosophy. While standard applications connect directly to a backend, I engineered an inline TLS inspection proxy that acts as an enforcement node. It terminates TLS handshakes, performs deep packet inspection on our custom binary protocol, evaluates zero-trust policies and device posture, runs real-time DLP to mask sensitive data like credit cards and API keys in-flight, and re-encrypts the session upstream to the server — providing complete visibility without compromising performance."